

SE Lab 4 - VibeCoding

Chat History

Project: Mastermind | Repository: SETAPESU26/20_mastermind

Conversation included up to successful completion and push of the project.

User

hey i have to do a SE project and i have to given a github repo with a few docs including a readme file a requirements txt file and 3 python files guide me what to do

Assistant

The lab is essentially: take a broken Python project, understand it, fix it using AI prompts, add the requested features, and show proper Git commits. The handout requires understanding the README, cloning/forking the repo, running the original code, recording a 10-second before video, fixing bugs, adding README features one by one, recording a 10-second after video, pushing to your own repo with separate commits, and uploading the videos, updated code, and exported AI chat history under Lab-4.

I asked for the GitHub repo link or the README, requirements.txt, and the three Python files so I could map out the exact tasks.

User

https://github.com/SETAPESU26/20_mastermind

Assistant

I checked the repo. The project is Mastermind, a terminal code-breaking game. The repo contains main.py, game.py, logic.py, and an effectively empty requirements.txt. It runs with:

```
python main.py
```

The README gives four tasks: fix duplicate-aware feedback, complete the game lifecycle, add difficulty modes, and improve history/input handling. I recommended doing one task at a time with a separate commit for each.

I also advised forking the college repo into the user's own GitHub account, cloning the fork, recording the before video before edits, then implementing and committing each task separately.

User

where should i run this

Assistant

Run the Git commands in a terminal on your laptop, easiest through VS Code: open VS Code -> Terminal -> New Terminal. Then run the clone command, cd into the project folder, and run the Python file.

```
git clone https://github.com/YOUR_USERNAME/20_mastermind.git
cd 20_mastermind
python main.py
```

User

I ran git clone but PowerShell says 'git is not recognized as the name of a cmdlet...'

Assistant

That means Git is not installed or not available in PATH. I told you to install Git for Windows, reopen VS Code, verify with:

```
git --version
```

Then clone the repository again.

User

yeah im in 20_mastermind folder

Assistant

Next I told you to run `dir`, confirm `README.md`, `requirements.txt`, `main.py`, `game.py`, and `logic.py` are present, then run the original program before changing anything.

```
python main.py
```

I reminded you to record the required 10-second before video first.

User

it says enter 10 digits from 1 to 6 but whatever i enter it is not taking

Assistant

I clarified that 10 is the number of turns, not the number of digits. The current game expects exactly four digits, each from 1 to 6. For example:

```
1234
```

I explained that inputs like '1 2 3 4', 123, 12345, 1789, or abcd would be rejected.

User

got it got it after getting the video what should i do

Assistant

I said to fork the repository into the user's own GitHub account before editing/pushing, because the lab requires pushing to the user's personal repo and not raising a PR to SETAPESU26. I asked the user to check the current remote with:

```
git remote -v
```

User

idk how to fork

Assistant

I guided the user to open the repository on GitHub, click Fork, choose their account as Owner, keep the name 20_mastermind, and click Create fork. Then I told them to run `git remote -v`.

User

`git remote -v` shows origin `https://github.com/SETAPESU26/20_mastermind` for fetch and push

Assistant

I explained that the local folder was still connected to the college repo. After creating the fork, I told the user to change origin to their own fork:

```
git remote set-url origin https://github.com/YOUR_USERNAME/20_mastermind.git
```

Then verify with `git remote -v` so both fetch and push point to the user's repo.

User

done

Assistant

Task 1: fix duplicate-aware feedback. I told the user to open `logic.py` and replace it with this implementation, which resolves exact matches first and removes matched symbols so duplicates are not counted twice:

```
def feedback(code, guess):
    exact = 0
    partial = 0

    remaining_code = []
    remaining_guess = []

    for c, g in zip(code, guess):
        if c == g:
            exact += 1
        else:
            remaining_code.append(c)
            remaining_guess.append(g)

    for g in remaining_guess:
        if g in remaining_code:
            partial += 1
            remaining_code.remove(g)

    return exact, partial
```

After testing, I told the user to commit Task 1 separately:

```
git add logic.py
git commit -m "Fix duplicate-aware feedback"
```

User

how to open logic.py in VS Code

Assistant

I said to press Ctrl+P, type logic.py, and press Enter, or open it from the Explorer panel on the left. Then Ctrl+A to replace the contents and Ctrl+S to save.

User

yeah done the code works

Assistant

I told the user to commit Task 1 and verify the working tree:

```
git add logic.py
git commit -m "Fix duplicate-aware feedback"
git status
```

User

should i record after the changes done to logic file

Assistant

No. I explained that the required final after video should be recorded only after all changes are complete. Keep the before video, finish Tasks 1-4, test everything, then record the final after video.

User

i got author identity unknow error

Assistant

I explained that Git did not know the commit author yet and gave these commands:

```
git config --global user.name "Spoorthy M"
git config --global user.email "YOUR_GITHUB_EMAIL"
```

Then retry git add and git commit.

User

done what to do next

Assistant

Task 2: complete the game lifecycle. I told the user to replace game.py with a version that adds max_turns, history, game_over, won, quit handling, invalid-input handling, clear win handling, and loss handling after all turns.

```
import random
from logic import feedback

class Mastermind:
    def __init__(self):
        self.code = [str(random.randint(1, 6)) for _ in range(4)]
        self.max_turns = 10
        self.history = []
        self.game_over = False
        self.won = False

    def run(self):
        print("Mastermind – enter four digits from 1 to 6.")

        while len(self.history) < self.max_turns and not self.game_over:
            turns_left = self.max_turns - len(self.history)
            guess = input(f"{turns_left} turns left > ")

            if guess.lower() == "q":
                print("Game ended.")
                self.game_over = True
                break

            if len(guess) != 4 or any(ch not in "123456" for ch in guess):
                print("Invalid guess. Enter exactly four digits from 1 to 6.")
                continue

            exact, partial = feedback(self.code, guess)
            self.history.append((guess, exact, partial))
            print(f"Exact: {exact} Partial: {partial}")

            if exact == 4:
                self.won = True
                self.game_over = True
                print("You won!")
                break

            if not self.won and not self.game_over:
                self.game_over = True
                print(f"You lost! The code was {''.join(self.code)}.")
```

I told the user to test q, a full loss, and a win, then commit:

```
git add game.py
git commit -m "Complete game lifecycle"
```

User

done

Assistant

Task 3: add difficulty modes. I provided a version of game.py with Easy, Medium, and Hard settings and a choose_difficulty method.

```
Easy -> 4 digits, symbols 1-4, 12 turns
Medium -> 4 digits, symbols 1-6, 10 turns
Hard -> 5 digits, symbols 1-8, 8 turns
```

I told the user to test all three modes and an invalid menu choice, then commit:

```
git add game.py
git commit -m "Add difficulty modes"
```

User

done

Assistant

Task 4: improve history and input handling. I provided the final game.py version with a readable Guess History display, whitespace trimming, exact-length validation, symbol-range validation, and invalid guesses that do not consume turns.

```
def show_history(self):
    if not self.history:
        return

    print("\nGuess History:")

    for i, item in enumerate(self.history, start=1):
        guess, exact, partial = item
        print(f"{i}. {guess} -> Exact: {exact}, Partial: {partial}")

    print()
```

I told the user to test inputs such as abc, 123, 9999, and 12 34 and confirm the turn count does not decrease for invalid input. Then commit:

```
git add game.py
git commit -m "Improve history and input validation"
```

User

done

Assistant

I said the coding part was complete and moved to final testing, the after video, and pushing. I recommended testing Easy/Medium/Hard, invalid input, valid input with history, q to quit, and ideally a win or full loss.

```
python main.py
```

Then I told the user to check status and push the main branch:

```
git status
git push origin main
```

User

git status shows branch main is ahead of origin/main by 4 commits, plus untracked __pycache__/. git push origin main completed successfully with main -> main.

Assistant

I confirmed that the push succeeded and that the four commits matched the four tasks. I explained that __pycache__/ is only generated temporary Python data and does not need to be uploaded.

I then said the coding and Git part was done. The remaining submission steps were to refresh the GitHub fork, record the 10-second after video if not already done, keep before.mp4 and after.mp4, export the chat history as PDF/doc, and place the required materials under Lab-4. I also reminded the user not to create a pull request to SETAPESU26.